

Nuxeo Platform Implementation & Customization Plan

Rebranding, System Customization, and Commercial Deployment Strategy

1. Executive Summary

This document provides a practical, step-by-step implementation plan for rebranding, customizing, and deploying the Nuxeo Platform as your own commercial Media Asset Management (MAM) product. It covers everything from initial setup to production deployment, with a focus on preserving seamless user experience.

The core philosophy of this plan is simple: Never edit Nuxeo core files directly.

All customizations must be managed cleanly through:

- Nuxeo Studio (low-code visual customization)
- Configuration properties (nuxeo.conf)
- Custom bundles/contributions
- Theme overrides

Following this standard ensures your custom layers remain intact across platform updates and vendor upgrades.

2. Rebranding Checklist

2.1 Quick Branding Overview

Nuxeo recommends dedicating **one hour to customize the core look and feel**: update the logo, adjust theme colors, and personalize the authentication login screen to establish your brand presence immediately.

2.2 Step-by-Step Branding Process

Step 1: Upload Your Logo to Resources

1. Log in to your Nuxeo instance as Administrator.
2. Navigate to **Nuxeo Studio** (accessible via the top header menu).
3. In Studio's left navigation menu, go to **Resources**.
4. Click **Upload** and select your primary logo asset (PNG format recommended).

Step 2: Create Your Branding Configuration

1. In Studio's left menu, click on the **Branding** section.
2. Select the **New** button.
3. In the **Feature ID** field, enter a unique identifier for your brand configuration (e.g., **MyBranding**).
4. Click **Next**.

Step 3: Configure the Login Screen

Setting	Action Required
Logo image	Select Choose resource and pick your uploaded logo asset
Background	Select a high-resolution background graphic or modern brand background color
Login button color	Set primary call-to-action button color to match your brand palette
Footer text	Remove default references and update with your corporate copyright notice
News boxes	Hide default news feeds or point to your internal announcements feed

Step 4: Apply Default Settings

- 1. Check the box labeled **"Set as default branding"**.
- 2. Click **Save** to confirm changes.

Step 5: Apply Branding to Main Application Pages

- 1. In the Branding workspace, navigate to the **Main Pages** tab.
- 2. Adjust visual properties:
 - Header background colors and container layout
 - Typography family and font weight hierarchy
 - Primary, secondary, and accent button styles
 - Top navigation menu bar aesthetics
- 3. Click **Save**.

2.3 Branding Verification Checklist

Verification Task	Target Result	Status
Login Page Logo	Custom logo displays clearly on authentication screen	☐ ▾
Header Logo	Brand logo appears in main top navigation header	☐ ▾
Color Consistency	Primary and secondary brand colors map correctly across application	☐ ▾
Background Assets	Updated login background renders correctly on all resolutions	☐ ▾

Verification Task	Target Result	Status
Platform Decoupling	Vendor "Nuxeo" text references removed from user interface	☐ ▾
Footer Information	Legal entity copyright and footer text updated	☐ ▾

3. Theme Customization

3.1 Built-in Themes

Nuxeo Web UI includes several out-of-the-box themes available under **User Settings**:

- **Default**: Standard interface look
- **Dark**: High-contrast dark interface
- **Light**: Clean, high-lightness interface
- **Garden**: Soft, natural color palette
- **Kawaii**: Accent-focused alternative layout

3.2 Creating a Custom Theme

Option 1: Using Nuxeo Studio (Recommended)

1. In Studio Designer, navigate to **UI → Themes**.
2. Click **New** and supply a unique theme identifier.
3. Define theme variables:
 - **Logo**: Upload header mark
 - **Background**: Define global background color or image
 - **Colors**: Assign primary, secondary, and accent colors
 - **Typography**: Select default application font family and sizing scale
4. Set the new creation as the **Default Theme** for your application deployment.

Option 2: Overriding Theme Resources (Advanced)

To adjust lower-level CSS parameters, deploy custom overriding theme packages./themes/{name}/

└─ theme.html # Style definitions for colors, fonts, CSS variables

└─ theme.js # Script definitions and visual behavior hooks

```
/* Example CSS Variable Overrides */
```

```
:root {  
  
  --nuxeo-app-font: 'Inter', system-ui, sans-serif;  
  
  --nuxeo-header-background: #0F172A;  
  
  --nuxeo-button-primary: #2563EB;  
  
  --nuxeo-primary-color: #2563EB;  
  
}
```

3.3 Adding Custom Header and Footer Component

To introduce custom branding bars or organizational notices into Nuxeo Web UI:

1. Locate your central `theme.html` file where visual tokens are defined.
2. Insert custom HTML element blocks designated for top headers and bottom utility footers.
3. Apply overriding layout styles to position elements within the Web UI slot hierarchy.

3.4 Overriding Default Styles (SCSS)

Default application styling relies on dynamic SCSS source files linked to style flavors.

themes/custom/theme.css

This configuration appends styles to `nuxeo\_sassCss`, guaranteeing proper order in style evaluation.

3.5 Theme Customization Guidelines

| Recommended Practices  | Prohibited Practices  |

| :--- | :--- |

| Leverage Nuxeo Studio for low-code changes | Never edit Nuxeo core installation files directly |

| Override themes via custom deployed bundles | Do not alter raw core files on the physical server |

| Utilize UI Slots for incremental features | Avoid replacing whole layouts when adding minor items |

| Validate themes in local dev containers first | Avoid deploying unvetted visual updates directly to production |

4. Core Feature Configuration

4.1 Digital Asset Management (DAM) Setup

The Digital Asset Management module provides structural document types for pictures, audio, and rich video assets.

Installation Sequence

1. In **Nuxeo Studio**, select the **DAM** package under **Application Definition**.

2. Activate primary asset document schemas (`Video`, `Picture`, `Audio`).

Capabilities Matrix

* **Import & Storage**: Fully integrated

* **Mimetype Identification**: Automated upon ingest

* **Thumbnails & Storyboards**: Native background generation

* **Metadata Extraction**: Automated (EXIF, IPTC, XMP, ID3)

* **Transcoding & Delivery**: Dynamic rendering, multi-format delivery, and automated watermarking

4.2 MXF Broadcast Format Support

MXF container support is fully functional within the platform infrastructure:

* **Mimetype**: `application/mxf`

* **Features**: Extraction, auto-proxies, storyboard extraction, watermarked delivery

* **Prerequisites**: Confirm native installation of `ffmpeg` and `ccextractor` binaries on host execution nodes.

4.3 Transcoding & Video Conversion Configuration

Video conversions are managed using XML extension declarations targeted at `videoConversions`.

```
```xml
```

```
<extension target="org.nuxeo.ecm.platform.video.service.VideoService"
point="videoConversions">
```

```
<videoConversion name="MP4 720p" converter="ffmpegToMP4" height="720">
```

```
<extension>mp4</extension>
```

```
</videoConversion>
```

```
</extension>
```

## 4.4 Storage Infrastructure (Blob Providers)

By default, binary files are managed on local disks using content-addressable storage (CAS) mechanics.

```
org.nuxeo.ecm.core.blob.binary.DefaultBinaryManager
/var/lib/nuxeo/data/binaries
```

> **Important:** Ensure isolated distinct storage paths for independent local binary providers. When connecting to S3 buckets, guarantee separate ``bucket_prefix`` paths per environment.

### ### 4.5 Archival & Cold Storage Strategy

The **Nuxeo Cold Storage** plugin transitions stale binary content to cost-efficient cloud storage classes (e.g., AWS S3 Glacier).

#### #### Prerequisites

1. Deploy ``nuxeo-coldstorage`` package.



2. Deploy and configure Amazon S3 Online Storage extension module.

#### Configuration Settings (`nuxeo.conf`)

```
``properties
```

```
Cloud Storage Credentials
```

```
nuxeo.aws.accessKeyId=YOUR_ACCESS_KEY_ID
```

```
nuxeo.aws.secretKey=YOUR_SECRET_KEY
```

```
nuxeo.aws.region=us-east-1
```

```
Primary S3 Storage Settings
```

```
nuxeo.s3storage.bucket=corporate-mam-media
```

```
nuxeo.s3storage.bucket_prefix=production/
```

```
Cold Storage Retention Policies
```

```
nuxeo.coldstorage.numberOfDaysOfAvailability.value.default=2
```

## 4.6 YouTube Publishing Integration

Publish video assets directly from the MAM system into target YouTube brand channels via API integration.

### Configuration Steps

1. Install the module via command line: `./bin/nuxeoctl mp-install nuxeo-mediapublishing`
2. Enable YouTube Data API v3 in Google Cloud Developers Console and create OAuth2 Credentials.
3. In Nuxeo Admin, navigate to **Cloud Services** → **Service Providers**, select **YouTube**, and insert your Client ID and Secret.

4. Test by opening a Video asset in DAM, selecting **Publish → YouTube**, selecting your channel, and confirming ingest.

## 5. Deployment Checklist

### 5.1 Pre-Deployment Preparation

Operational Task	Verification Command / Detail	Status
License Verification	Confirm Apache 2.0 notices remain intact	<input type="checkbox"/> ▾
Online Services Setup	Link Nuxeo Studio account credentials	<input type="checkbox"/> ▾
Java Runtime	JDK 21 installed ( <code>java -version</code> )	<input type="checkbox"/> ▾
Build Tooling	Apache Maven 3.9.6+ installed ( <code>mvn -version</code> )	<input type="checkbox"/> ▾
Container Engine	Docker Engine active ( <code>docker --version</code> )	<input type="checkbox"/> ▾
Kubernetes Tools	Install <code>kubectl</code> and <code>helm</code>	<input type="checkbox"/> ▾

### 5.2 Development Environment Setup

Action Step	Execution Command / URL	Status
Fork Core Repository	<a href="https://github.com/nuxeo/nuxeo">https://github.com/nuxeo/nuxeo</a>	<input type="checkbox"/> ▾

Action Step	Execution Command / URL	Status
Clone Workspace	<code>git clone https://github.com/YOUR_ORG/nuxeo.git</code>	<input type="checkbox"/> ▾
Build Custom Engine	<code>mvn -DskipTests install -Paddons,distrib</code>	<input type="checkbox"/> ▾
Local Container Test	<code>docker run --name nuxeo-dev -p 8080:8080 -e NUXEO_DEV=true docker.packages.nuxeo.com/nuxeo/nuxeo:2025</code>	<input type="checkbox"/> ▾

## 5.3 Containerized Execution (Docker)

### Development Instance (Hot Reloading Active)

```
h
docker run --name nuxeo-dev -p 8080:8080
-v /var/nuxeo/data:/var/lib/nuxeo
-v /var/nuxeo/log:/var/log/nuxeo
-e NUXEO_DEV=true
docker.packages.nuxeo.com/nuxeo/nuxeo:2025
```

#### Production Container Runtime

```
```bash
```

```
docker run --name nuxeo-prod -p 8080:8080 \
```

```
-v /mnt/nuxeo/data:/var/lib/nuxeo \
```

```
-v /mnt/nuxeo/log:/var/log/nuxeo \
```

```
-v /mnt/nuxeo/conf:/etc/nuxeo/conf.d \
```

```
docker.packages.nuxeo.com/nuxeo/nuxeo:2025
```

5.4 Kubernetes Deployment with Helm

Add Nuxeo Helm Repository

```
helm repo add nuxeo
```

```
https://packages.nuxeo.com/repository/helm-releases-public/
```

Install Helm Release

```
helm install mam-platform nuxeo/nuxeo -f values.yaml
```

Production Values File (values.yaml)

```
replicaCount: 2
```

```
nuxeo:
```

```
  conf:
```

```
    nuxeo.db.type: "postgresql"
```

```
    nuxeo.db.host: "postgres-cluster"
```

```
    nuxeo.db.name: "nuxeo_mam"
```

```
    nuxeo.db.user: "mam_admin"
```

```
    nuxeo.s3storage.bucket: "prod-mam-store"
```

nuxeo.s3storage.bucket_prefix: "mam/"

nuxeo.jvm.args: "-Xms4096m -Xmx8192m"

postgresql:

enabled: true

postgresqlDatabase: "nuxeo_mam"

elasticsearch:

enabled: true

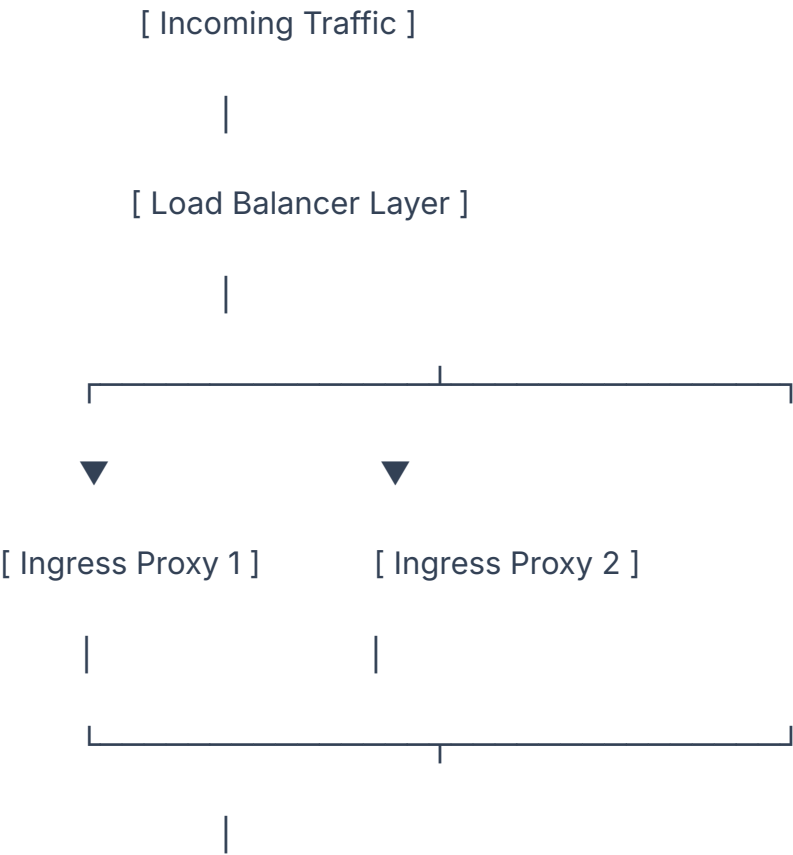
replicas: 3

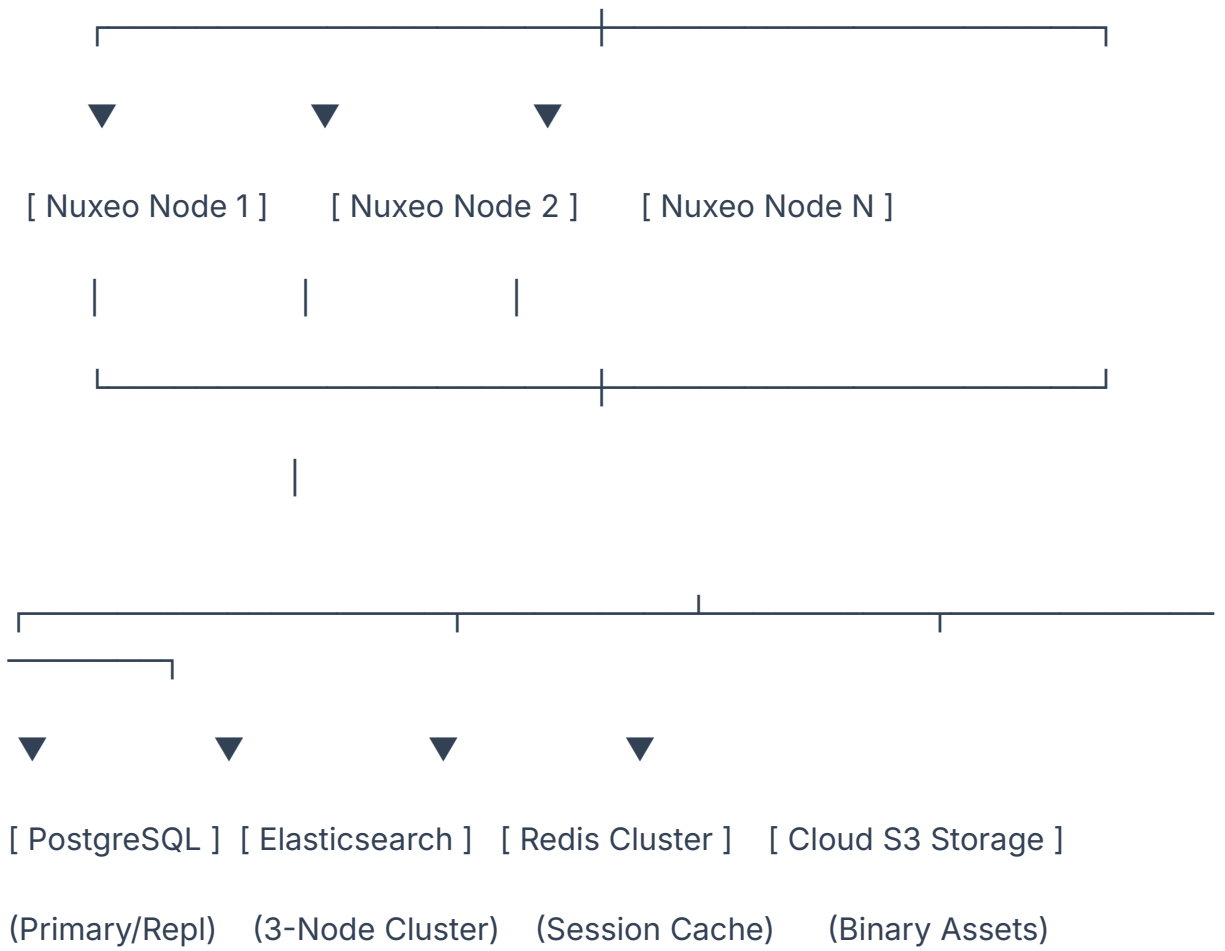
5.5 Production Readiness Checklist

Infrastructure Area	Requirement	Status
Database Layer	PostgreSQL (replace embedded H2)	☐ ▾
Blob Storage	Amazon S3 or S3-compatible API (MinIO)	☐ ▾
Search Engine	Multi-node Elasticsearch cluster (Minimum 3 nodes)	☐ ▾
Cache & Messaging	Redis cluster (Minimum 3 nodes)	☐ ▾
Memory Sizing	JVM Heap set between 4GB and 16GB minimum	☐ ▾

Infrastructure Area	Requirement	Status
Edge Ingress	Reverse proxy (Nginx / HAProxy) with SSL termination	☐ ▾
Backup Strategy	Automated snapshot schedule for database and blob indexes	☐ ▾
Observability	Metrics scraped via Prometheus & dashboards in Grafana	☐ ▾

5.6 High-Availability Cluster Architecture





Cluster Component	Responsibility	Minimum HA Node Count
Load Balancers	Ingress distribution and health checks	2
Reverse Proxies	Security, SSL termination, and caching	2
Nuxeo Application Nodes	Business logic & asset processing	2+
Database Engine	Asset transactional metadata state	2 (Primary + Standby)

Cluster Component	Responsibility	Minimum HA Node Count
Search Index	Deep full-text & metadata indexing	3
Cache Layer	In-memory operations & queue management	3

6. Customization Rules

6.1 The Golden Rule

Never edit Nuxeo core files directly.

Always isolate changes within standard Extension Points, Nuxeo Studio configurations, `nuxeo.conf` properties, and dedicated deployment bundles (`/themes/{name}/`).

6.2 Development Do's and Don'ts

Approved Approaches 	Prohibited Actions 
Manage UI layout via Studio Designer	Modifying base system distribution binaries
Override themes via isolated bundle contributions	Editing legacy JSF UI assets directly
Add extensions via modular extension points	Overwriting base XML contributions without testing
Validate changes in staging environments	Direct deployment of uncompiled code to production

6.3 Framework Deprecations

- The **JSF UI framework** is deprecated.
- All new development and user interface extensions must target **Nuxeo Web UI** (Web Components, Polymer, HTML5/CSS3) and utilize **REST APIs**.

7. Testing & Quality Assurance Plan

7.1 Functional Verification

Test Scenario	Steps / Criteria	Target Result	Status
Asset Ingestion	Upload high-res MP4 and MXF assets	Successful file write, automated metadata display	☐ ▾
Proxy Generation	Process primary media asset	Playable H.264 web proxy built within 5 min	☐ ▾
Video Transcoding	Execute job converting MXF container to MP4	Transcoding job completes without errors	☐ ▾
Third-Party Publishing	Trigger YouTube integration pipeline	Asset published with metadata to channel	☐ ▾
Search Indexing	Search via new metadata properties	Instant returning of newly indexed assets	☐ ▾
Asset Lifecycle	Execute archive policy on targeted asset	Asset payload relocated to Cold Storage bucket	☐ ▾

7.2 User Experience (UX) Verification

Test Scenario	Target Standard	Status
Logo Visibility	Branding logo rendered clearly across all screen sizes	☐ ▾
Palette Application	Accent colors updated on primary buttons and indicators	☐ ▾
Responsive Layout	Layout adjusts gracefully from 1080p desktop down to mobile viewports	☐ ▾
Navigation Flow	Custom menu links and navigation slots respond smoothly	☐ ▾

7.3 Performance Benchmarks

Metric	Threshold Target	Status
Authentication Speed	Initial load < 3.0 seconds	☐ ▾
UI Response Time	View render < 3.0 seconds	☐ ▾
Ingestion Bandwidth	100MB file ingest < 2.0 minutes	☐ ▾
Search Latency	Full-text query return < 3.0 seconds	☐ ▾

Metric	Threshold Target	Status
Concurrency Target	Maintain performance at 100+ active users per application node	☐ ▾

8. Troubleshooting Guide

8.1 Common Operational Issues

Symptom	Primary Cause	Resolution Action
Custom branding missing	Configuration not flagged active	Select "Set as default branding" inside Studio
Theme styles not updating	Deployment cache stale	Force re-deploy theme bundle and clear browser cache
YouTube publish failure	OAuth2 token expired	Re-authenticate client credentials inside Cloud Services
Transcoding jobs failing	Missing media binaries	Verify ffmpeg and ccextractor installation paths
Cold storage errors	AWS S3 permission block	Verify IAM policy permissions on target S3 bucket

8.2 Standard Debugging Sequence

1. **Inspect Application Logs:** Review `/var/log/nuxeo/server.log` or execute `docker logs container_id`.
2. **Validate Environment Settings:** Review `nuxeo.conf` properties to ensure syntax correctness.
3. **Isolate Customizations:** Temporarily un-deploy custom bundles to determine if issue stems from core platform or custom logic.
4. **Consult Documentation:** Reference official developer resources at <https://doc.nuxeo.org>.

9. Approval Sign-off

Project Role	Designated Person	Approval Date	Signature
Lead Engineer	 Person	 Date	
DevOps Engineer	 Person	 Date	
QA Lead	 Person	 Date	
Product Manager	 Person	 Date	
Client Representative	 Person	 Date	